

CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA PAULA SOUZA
FACULDADE DE TECNOLOGIA DE SOROCABA JOSÉ CRESPO
GONZALES

JOÃO ROBERTO MEIRINHO NUNES

ARQUITETURA DE SOFTWARE

Programação Web

RA: 0030482511016

SOROCABA – SP

2026

Sumário

1. Definições formais de Arquitetura de Software	2
Decomposição	3
Pilares da Programação Orientada a Objetos	4
2. Tipos de arquitetura de software	4
Monolito Tradicional	5
Monolito Modular	5
Microsserviços	6
3. Padrões de projeto e protocolos de comunicação	6
Padrões de Projeto	8
Domain-Driven Design (DDD) e Bounded Context	8
Backend for Frontend (BFF)	8
Infraestrutura	8
Service Discovery e API Gateway	9
Protocolos de comunicação	9
REST, GraphQL e gRPC	9
4. Escolha da arquitetura e tomada de decisão	10
5. Tendências tecnológicas	11
Governança arquitetural	11
Implementação técnica	11
Exemplo ArchUnit (Java):	11
Exemplo NetArchTest (.NET):	12
Micro frontends	12
Decomposição e separação por domínios	13
Computação Serverless e FaaS (Function as a Service)	13
Arquitetura de Gêmeos Digitais (Digital Twins)	13
Inteligência artificial como microsserviços	14
Referência bibliográficas	15

1. Definições formais de Arquitetura de Software

A Arquitetura de Software constitui o alicerce estratégico do desenvolvimento de sistemas, representando onexo causal entre as necessidades de negócio e a viabilidade técnica. A padronização terminológica não é meramente um exercício semântico, mas um imperativo para a maturidade da Engenharia de Software; a ausência de rigor conceitual gera o que a literatura contemporânea identifica como uma lacuna no corpo de conhecimento (gap in the body of knowledge), dificultando a migração sistemática de sistemas legados.

No plano formal, a norma ISO/IEC/IEEE 42010:2011 define arquitetura como a organização fundamental de um sistema incorporada em seus componentes, seus relacionamentos e os princípios que guiam seu design e evolução. Esta visão é complementada pela tríade Bass, Clements & Kazman, que a descreve como o conjunto de estruturas necessárias para raciocinar sobre o sistema, focando em elementos e propriedades externas. Contudo, na perspectiva de Richards & Ford, a arquitetura transcende a estrutura, abrangendo as características de arquitetura, decisões de design e princípios fundamentais. Tais decisões são críticas pois determinam diretamente o desempenho, a manutenção, a segurança e o custo operacional.

Decomposição

A decomposição é a ferramenta estratégica para gerenciar a complexidade cognitiva, transformando problemas intratáveis em módulos gerenciáveis. É imperativo notar que a escolha do paradigma de decomposição é o fator determinante para a maleabilidade do software. Segundo dados de Ponce et al., aproximadamente 90% das aplicações monolíticas que passam por processos de migração para microsserviços foram originalmente escritas sob o paradigma orientado a objetos, o que ressalta a centralidade deste modelo na engenharia moderna.

Historicamente, dois paradigmas contrastam em rigor e aplicação:

1. **Decomposição Funcional (ou Estruturada):** Baseia-se no princípio de "dividir para conquistar", focando em processos e algoritmos. Utiliza um processo de Refinamento sistemático, onde funções de alto nível são sucessivamente quebradas até atingirem o estado de funções primitivas. É uma abordagem essencialmente baseada em verbos.
2. **Decomposição Orientada a Objetos:** Foca no domínio do problema, identificando entidades e as mensagens trocadas entre elas. Conforme o Adapter Microservices Pattern, esta é uma transição de uma abordagem

baseada em verbos para uma abordagem baseada em substantivos (entidades).

A superioridade da decomposição orientada a objetos reside na sua capacidade de encapsular volatilidade. Enquanto a decomposição funcional frequentemente gera dependências estáticas rígidas, a orientação a objetos permite que cada componente evolua de forma independente, definindo-se por seus atributos e comportamentos únicos. Essa flexibilidade é o que permite a transição sustentável para arquiteturas distribuídas, onde a autonomia modular é o requisito fundamental para a agilidade de deploy.

Pilares da Programação Orientada a Objetos

O surgimento do princípio de Information Hiding, proposto por David Parnas em 1972, estabeleceu o critério definitivo para a modularização. Parnas postulou que a eficácia de um módulo não reside no que ele exhibe, mas no que ele oculta. Sua máxima é clara: o design deve "permitir que módulos sejam remontados e substituídos sem a remontagem do sistema global" (Parnas, 1972). Este conceito é o precursor direto da visão de Sam Newman sobre microsserviços como "caixas pretas" que ocultam detalhes internos de implementação.

Este princípio é operacionalizado através dos pilares da Orientação a Objetos:

- **Abstração:** Extração de aspectos essenciais de uma entidade, suprimindo detalhes secundários para reduzir a carga cognitiva.
- **Encapsulamento:** Mecanismo de proteção que restringe o acesso direto aos dados, expondo apenas interfaces por meio de métodos públicos.
- **Herança:** Promoção de reusabilidade e extensibilidade hierárquica. Sua validade arquitetural é garantida pelo Princípio de Liskov (LSP), que exige que uma subclasse seja substituível por sua classe base sem alterar a correção do sistema — um requisito vital para a autonomia modular.
- **Polimorfismo:** Capacidade de objetos de diferentes tipos responderem à mesma interface, permitindo a substituição de componentes em tempo de execução.

A aplicação estrita do Princípio de Parnas e do LSP reduz o tempo de desenvolvimento ao permitir que equipes trabalhem em paralelo com comunicação mínima. Ao ocultar as "decisões de design difíceis ou propensas a mudanças", o arquiteto isola o impacto de alterações, criando o ambiente necessário para a evolução contínua.

2. Tipos de arquitetura de software

A seleção de um modelo arquitetural transcende a mera escolha técnica; constitui-se como o alicerce estratégico indispensável para a sustentabilidade e escalabilidade de sistemas de software de alta complexidade. Na Engenharia de Software contemporânea, a transição de paradigmas — partindo dos fundamentos clássicos de Pressman e Sommerville até as necessidades de agilidade identificadas por Cintra et al. — exige um retorno às teorias seminais para compreender os desafios modernos.

O pilar central dessa discussão reside no "Princípio de Ocultação de Informação" (Information Hiding), proposto por David Parnas em 1972. Parnas argumentou que a decomposição de um sistema em módulos independentes deve visar a redução da complexidade através de interfaces claras, tratando componentes como "caixas-pretas". É imperativo notar que a "componentização via serviços" defendida por Sam Newman e Martin Fowler é a sucessora espiritual direta do conceito de Parnas. O que Newman define como "ocultar detalhes de implementação interna" nada mais é do que a aplicação do Information Hiding em uma fronteira de implantação distribuída. O conceito de encapsulamento permanece inalterado; apenas o limite da "caixa-preta" migrou do módulo em memória para o serviço independente. Esta compreensão é basilar para avaliar o monólito tradicional e suas evoluções.

Monolito Tradicional

Historicamente, o monólito consolidou-se pela sua onipresença em projetos de pequeno e médio porte. Conforme sugerido por Velepucha & Flores, sua importância estratégica reside na baixa barreira de entrada para equipes reduzidas e na simplicidade de gerenciamento de um único executável e uma base de código unificada.

Arquiteturalmente, o monólito beneficia-se de comunicações intraprocesso (in-memory), que garantem latência desprezível, e da facilidade em manter transações ACID (Atomicidade, Consistência, Isolamento e Durabilidade) em um banco de dados centralizado. Contudo, o escalonamento impõe desafios severos. Operacionalmente, o monólito pode ser escalado via WebGarden (instanciando múltiplos processos de trabalho no mesmo servidor) ou via WebFarm (replicando o monólito em diversos servidores através de um balanceador de carga). No entanto, ambos os métodos carecem de granularidade: para escalar uma funcionalidade específica, é obrigatório replicar o bloco inteiro, gerando ineficiência de recursos e lock-in tecnológico. A transição para o monólito modular surge, portanto, como uma resposta planejada para organizar essa unidade sem introduzir prematuramente a complexidade da rede.

Monolito Modular

O monólito modular atua como o "caminho do meio", focando na organização interna rigorosa como preparação para a escalabilidade seletiva. Sua estruturação é fundamentada no Domain-Driven Design (DDD), utilizando Bounded Contexts (Contextos Delimitados) para definir fronteiras lógicas claras. Aqui, a alta coesão e o baixo acoplamento são alcançados através de dependências explícitas entre módulos que, embora compartilhem o mesmo ciclo de implantação, operam com autonomia lógica.

Neste modelo, a comunicação via eventos em memória evita o overhead de rede, preservando a performance enquanto adota a "flexibilidade incremental" de Parnas. Estrategicamente, o monólito modular é o ponto de partida ideal para a aplicação do Strangler Application Pattern (Padrão Estrangulador). Através das etapas de Transform, Coexist e Remove, a organização pode migrar funcionalidades críticas do monólito para serviços independentes de forma faseada, mitigando riscos operacionais. Quando a barreira da implantação independente torna-se um gargalo para a agilidade do time, a transição para microsserviços consolida-se como o próximo passo evolutivo.

Microsserviços

Situada no ápice da flexibilidade arquitetural, a arquitetura de microsserviços, conforme definida por Fowler e Newman, promove a agilidade extrema através da implantação independente. Seus pilares — escalabilidade seletiva, heterogeneidade tecnológica e isolamento de falhas — permitem que organizações como Netflix e Amazon operem em escalas globais.

A mediação entre os serviços e os clientes é gerida por padrões como o API Gateway e o Backend for Frontend (BFF), garantindo o desacoplamento da interface. Contudo, a complexidade distribuída impõe um trade-off crítico na gestão de dados. A transição do modelo ACID para a consistência eventual (BASE) exige que o arquiteto adote a desnormalização de dados como um requisito técnico essencial; cada serviço deve possuir sua própria base para garantir autonomia, o que demanda padrões como o Saga Pattern para gerenciar transações distribuídas via compensação. Este custo de governança e a necessidade de observabilidade avançada devem ser pesados contra os ganhos de agilidade.

3. Padrões de projeto e protocolos de comunicação

A evolução das arquiteturas de software, partindo do modelo monolítico em direção aos microsserviços, reflete a necessidade premente de agilidade organizacional e escalabilidade técnica em sistemas complexos. Enquanto o

monólito, conforme discutido por Velepucha e Flores, caracteriza-se por um único executável onde os módulos não podem ser executados de forma independente, a arquitetura de microsserviços propõe uma aplicação distribuída em que cada unidade de negócio é autônoma. Esta transição é motivada pela busca de uma escalabilidade granular e pela capacidade de resposta rápida às demandas de mercado, permitindo que as organizações modernizem aplicações legadas sem a rigidez de uma base de código única e tecnologicamente homogênea.

A base conceitual da decomposição de sistemas modernos encontra eco no princípio de Information Hiding introduzido por David Parnas em 1972. Parnas argumentava que a modularização deveria visar a minimização das "dependências estáticas" entre as partes, permitindo que módulos fossem alterados ou substituídos sem afetar o sistema como um todo. Esta visão acadêmica é a precursora direta dos preceitos de Martin Fowler e Sam Newman na era dos microsserviços.

Princípio	Autor	Aplicação em Microsserviços
Information Hiding	David Parnas (1972)	Módulos como "caixas pretas", permitindo substituição independente e redução de dependências estáticas.
Componentização via Serviços	Martin Fowler	Uso de serviços independentes em vez de bibliotecas locais, facilitando o deploy e a escalabilidade individual.
Ocultação de Detalhes	Sam Newman	Garantia de que a implementação interna (ex: banco de dados) não seja exposta, preservando a autonomia.

A correlação entre a "Componentização via Serviços" (Fowler) e a "Ocultação de Detalhes de Implementação" (Newman) é o que mitiga o risco de falhas em cascata. Sob a ótica de um arquiteto sênior, a principal vantagem pragmática reside na redução do overhead de comunicação entre equipes. Como Parnas já indicava, tempos de desenvolvimento curtos são alcançados quando os módulos permitem o trabalho em grupo com o mínimo de comunicação interdependente. Em larga escala, o isolamento garantido por essas fronteiras

assegura que a falha de um serviço não comprometa a disponibilidade global, sustentando a resiliência do ecossistema.

A eficácia dessa decomposição, contudo, exige uma metodologia rigorosa para definir as fronteiras dos serviços, o que torna o Domain-Driven Design um pilar indispensável.

Padrões de Projeto

Alinhar o design de software aos domínios de negócio é um requisito fundamental para evitar o surgimento de "monólitos distribuídos". A falta de clareza nos limites dos serviços compromete a autonomia das equipes e gera acoplamentos desnecessários que anulam os benefícios da arquitetura distribuída.

Domain-Driven Design (DDD) e Bounded Context

Os conceitos de Eric Evans no Domain-Driven Design fornecem as ferramentas estratégicas para essa organização. O Bounded Context (Contexto Delimitado) é vital para garantir alta coesão e baixo acoplamento, permitindo que o sistema seja modelado em torno de capacidades de negócio. Academicamente, é importante notar a conexão intrínseca entre o DDD e o Princípio da Responsabilidade Única (Single Responsibility Principle), onde cada microsserviço foca em resolver um problema de negócio específico e bem delimitado, transicionando de uma visão meramente técnica para uma arquitetura baseada em domínios funcionais.

Backend for Frontend (BFF)

Com a fragmentação do backend, o padrão BFF surge como uma solução para otimizar a comunicação com diferentes clientes (web vs. mobile). Ele atua desacoplando o frontend da complexidade interna da malha de serviços. Tecnicamente, o BFF mitiga a latência percebida ao consolidar múltiplas chamadas de microsserviços em uma única resposta otimizada para a interface específica, reduzindo o processamento necessário no lado do cliente.

O principal trade-off na adoção de BFFs reside na complexidade de gerenciar múltiplos pontos de entrada em comparação com a flexibilidade de evolução. Embora aumente o número de componentes a serem monitorados, o padrão permite que cada interface evolua seu contrato de dados sem impactar as demais. Para um arquiteto, o custo operacional de manutenção de múltiplos BFFs é compensado pela agilidade em entregar experiências de usuário personalizadas e de alta performance.

Para sustentar essa estrutura dinâmica, é necessário introduzir mecanismos que gerenciem a volatilidade do tráfego e a descoberta de serviços.

Infraestrutura

Em ambientes nativos em nuvem, a volatilidade de endereços IP é uma constante. Instâncias de microsserviços são efêmeras, surgindo e desaparecendo conforme a demanda de escalabilidade e saúde do sistema, o que exige uma orquestração de tráfego automatizada.

Service Discovery e API Gateway

O Service Discovery resolve o problema da localização dinâmica através do Service Registry, onde cada serviço registra sua presença e emite sinais de vida (heartbeats). O API Gateway, por sua vez, centraliza o acesso ao sistema, servindo como o ponto de entrada único para o tráfego externo. Suas responsabilidades críticas incluem:

- **Autenticação e Autorização:** Centralização do controle de acesso.
- **Roteamento de Requisições:** Encaminhamento para as instâncias corretas via balanceador de carga.
- **Segurança:** Implementação de protocolos como mTLS e tokens de segurança.
- **Balanceamento de Carga:** Distribuição eficiente do tráfego norte-sul.

A centralização via API Gateway simplifica a governança, mas introduz um potencial ponto único de falha (Single Point of Failure - SPOF). A mitigação desse risco deve basear-se em conceitos consolidados de alta disponibilidade, como o uso de instâncias redundantes e load balancers. Conceitualmente, isso evolui de estratégias como WebFarms (escalonamento vertical/distribuído em servidores) e WebGardens (múltiplos processos em uma mesma máquina), aplicando-as agora em nível de container para garantir que a falha de um gateway não interrompa a operação.

Protocolos de comunicação

A escolha do protocolo impacta diretamente a performance inter-serviços (leste-oeste) e a comunicação com clientes externos (norte-sul), devendo alinhar-se ao princípio da heterogeneidade tecnológica.

REST, GraphQL e gRPC

Embora a literatura clássica enfatize o uso de mensagens leves via REST (HTTP/JSON) como o padrão para "Smart Endpoints", o cenário moderno expandiu as opções:

- **REST:** Onipresente e fácil de implementar, embora sofra com over-fetching. Baseia-se no modelo stateless e cache HTTP.
- **GraphQL:** Extensão moderna que resolve o problema de busca orientada ao cliente via query selectors, permitindo que o frontend solicite exatamente o que precisa, embora aumente a complexidade no servidor.
- **gRPC:** Focado em alta performance para comunicações internas. Utiliza HTTP/2 e serialização binária (Protocol Buffers), reduzindo drasticamente a latência e o overhead comparado à serialização textual do JSON.

4. Escolha da arquitetura e tomada de decisão

A arquitetura de software não deve ser compreendida como uma busca teleológica pela "melhor" solução universal, mas sim como uma disciplina rigorosa de escolhas críticas e compromissos técnicos. No exercício da engenharia de sistemas complexos, a seleção de um estilo arquitetural é o processo de equilibrar requisitos frequentemente conflitantes — como a celeridade no Time-to-Market versus a escalabilidade extrema. A maturidade de um estrategista manifesta-se na capacidade de discernir que a eficácia de uma arquitetura é contingente ao contexto organizacional e aos atributos de qualidade exigidos pelo domínio do problema.

Ecoando a máxima de Mark Richards e Neal Ford, "tudo em arquitetura de software é um trade-off". Esta premissa científica estabelece que para cada ganho em um atributo (como a resiliência dos microsserviços), há invariavelmente um custo associado (como o overhead operacional e a complexidade de rede). A fuga de "modismos" tecnológicos — mimetismos industriais que priorizam tendências passageiras em detrimento da estabilidade — é imperativa para a sustentabilidade do projeto. Conforme discutido por Cintra et al., a escolha deve fundamentar-se em parâmetros técnicos e de negócio, evitando a adoção de microsserviços por mera pressão estética ou de mercado.

Um conceito central para a sistematização da decisão é o Quantum Arquitetural. Definido como a menor unidade de arquitetura que possui alta coesão funcional, ele inclui todos os componentes necessários para operar de forma independente, abrangendo persistência de dados e dependências de rede. Essencialmente, o Quantum delimita a Integridade Transacional do sistema; além de suas fronteiras, a consistência eventual assume o protagonismo, exigindo mecanismos complexos de coordenação.

O impacto estratégico da correta identificação do Quantum reside na sua função como delimitador técnico para a proteção contra o acoplamento excessivo. Ao isolar essas unidades, o estrategista pode aplicar diferentes níveis de

escalabilidade e segurança, mitigando falhas em cascata. Para transcender a intuição técnica, é necessário aplicar o procedimento sistemático de avaliação fundamentado na metodologia de Cintra et al.

5. Tendências tecnológicas

A complexidade inerente aos ecossistemas de software contemporâneos exige uma transição paradigmática na forma como a integridade dos sistemas é preservada. Observa-se que a governança arquitetural evoluiu de modelos de revisão estáticos e esporádicos para abordagens contínuas e automatizadas. Conforme preconizado pela literatura, essa mudança é imperativa para sustentar a modularidade e evitar a degradação estrutural em ciclos de entrega acelerados. A governança contínua não atua apenas como um filtro de qualidade, mas como um mecanismo estratégico que garante que a evolução do sistema permaneça fiel aos seus princípios fundacionais, assegurando a sustentabilidade técnica e o valor de negócio a longo prazo.

Governança arquitetural

A simbiose entre as práticas de DevOps e a arquitetura de software redefine a manutenção da visão sistêmica. Baseando-se nos conceitos de Richards & Ford (2020) e nos princípios de modularização de David Parnas (1972), a governança automatizada emerge como o principal mecanismo de defesa contra a erosão arquitetural. Esta erosão, frequentemente causada por pressões de entrega que resultam em acoplamentos indevidos, é mitigada quando as diretrizes de design são codificadas e validadas em tempo de compilação ou execução.

Neste contexto, a aplicação de fundamentos de engenharia de software é vital. O Princípio de Substituição de Liskov, destacado no contexto diretivo como essencial para validar a correção de hierarquias de herança, deve ser verificado automaticamente para garantir que extensões de classes não alterem o comportamento esperado do sistema. Além disso, a governança contínua deve ser rigorosa na identificação de antipadrões, como o Shared Data Microservice Design Pattern. Embora aceitável em fases de transição, a manutenção prolongada de um repositório de dados compartilhado por múltiplos serviços viola a autonomia e o isolamento preconizados por Fowler e Newman, devendo ser alvo de alertas automáticos pelas funções de aptidão.

Implementação técnica

As Architectural Fitness Functions são testes automatizados que quantificam a conformidade do sistema com seus objetivos arquiteturais. Elas impõem regras de isolamento, como a proibição de acesso direto da camada de apresentação à camada de dados, respeitando a integridade das camadas lógicas.

Exemplo ArchUnit (Java):

Java

```
// Garante o isolamento de camadas e proíbe acesso  
direto ao repositório  
  
classes().that().resideInAPackage("..ui..")  
  
.should().onlyBeAccessed().byAnyPackage("..ui..")  
.andShould().notAccessClassesThat().resideInAPackage("..data.  
repository..");
```

Exemplo NetArchTest (.NET):

CSharp

```
// Impõe que a camada de serviços não possua  
dependências de infraestrutura  
  
var result = Types.InCurrentDomain()  
  
.That().ResideInNamespace("Application.Services")  
  
.ShouldNot().HaveDependencyOn("Infrastructure.Data")  
  
.GetResult();
```

Micro frontends

O conceito de Micro Frontends representa a evolução natural da modularização, estendendo os benefícios dos microsserviços para a experiência do usuário. Conforme as contribuições de Geers (2020), Mezzalana (2021) e Tilak et al. (2020), essa abordagem decompõe a interface em módulos autônomos, permitindo que cada fragmento visual seja gerido de forma independente.

Decomposição e separação por domínios

A decomposição do frontend em módulos autônomos correlaciona-se diretamente à organização de times em squads verticais focadas em domínios de negócio. Ao aplicar o Domain-Driven Design (DDD) para delimitar contextos, as equipes assumem a responsabilidade completa sobre uma funcionalidade, desde o backend até o componente visual renderizado. Essa estrutura promove a independência operacional e minimiza as dependências estáticas entre partes do sistema, ecoando os princípios de Parnas sobre a ocultação de informação para aumentar a flexibilidade.

Computação Serverless e FaaS (Function as a Service)

A consolidação de recursos de computação em nuvem (Compute Resource Consolidation) atingiu seu ápice com o paradigma Serverless, que redefine a responsabilidade sobre a infraestrutura. Neste modelo, a abstração é total, permitindo que o arquiteto se concentre exclusivamente na lógica de execução efêmera.

O modelo Function-as-a-Service (FaaS) opera sob demanda, processando funções stateless acionadas por eventos. Diferente das arquiteturas monolíticas que dependem de configurações de WebGarden (múltiplos processos em um único servidor) para otimização de recursos, o FaaS transfere a orquestração e a auto-escalabilidade para o provedor de nuvem. Isso otimiza custos operacionais ao cobrar apenas pelo tempo efetivo de execução, eliminando o desperdício de recursos ociosos inerentes aos servidores tradicionais.

Essa escalabilidade extrema é o requisito fundamental para suportar o processamento em tempo real exigido pela arquitetura de Gêmeos Digitais.

Arquitetura de Gêmeos Digitais (Digital Twins)

O conceito seminal de Gêmeos Digitais de Grieves (2002) define a convergência entre o mundo físico e o virtual através de modelos dinâmicos alimentados por dados em tempo real. A arquitetura de um DT exige um rigor superior ao de sistemas CRUD tradicionais, pois a fidelidade da sincronia é crítica para a tomada de decisão.

Com base nos princípios de microsserviços e IoT, as arquiteturas para DTs integram padrões como Layered, SOA e, fundamentalmente, Publish-Subscribe

para o processamento de streams de sensores. De acordo com a ISO 25010, os atributos críticos incluem:

- **Manutenibilidade:** A modularidade é essencial para atualizar modelos de simulação sem impactar o monitoramento físico.
- **Eficiência de Desempenho:** A baixa latência é mandatória para evitar o "communication delay" que tornaria o gêmeo virtual obsoleto em relação ao estado físico.
- **Interoperabilidade:** O uso de protocolos como MQTT e OPC UA, aliados a modelos semânticos (RDF/OWL), garante a integração de dados heterogêneos. O MQTT, especificamente, destaca-se por sua eficiência em cenários de IoT, mitigando o network overhead identificado como desvantagem em sistemas distribuídos tradicionais.

Inteligência artificial como microsserviços

A integração de IA em ecossistemas distribuídos, operando sob a infraestrutura de redes 5G e edge/fog computing, marca a nova fronteira da autonomia sistêmica. O paradigma AI as a Microservice (AIMS), conforme proposto por Lee, Um & Choi, permite que capacidades preditivas sejam consumidas de forma modular.

O desafio do AIMS reside na latência e nas severas restrições de recursos físicos na borda. Diferente da IA centralizada na nuvem, a IA na borda deve lidar com limitações de CPU e memória. Para garantir a segurança e a eficiência, aplicam-se estratégias de particionamento e posicionamento inteligente de contêineres e microsserviços de aprendizado de máquina (Al-Doghman et al.). Essa abordagem minimiza o trânsito de dados sensíveis para a nuvem pública, fortalecendo a privacidade.

Em conclusão, a convergência entre governança contínua, descentralização de interfaces e inteligência na borda reitera a necessidade de diretrizes rigorosas e automatizadas, assegurando que a complexidade tecnológica não se torne um obstáculo, mas um catalisador para a inovação resiliente.

Referência bibliográficas

CINTRA, Lucas Rodrigues; MARTINIANO, Pedro Paulino; BORGES, Leandro. Análise comparativa entre arquiteturas de software: monolítica, modular e microsserviços. RESIGeT - Revista Eletrônica de Sistemas de Informação e Gestão Tecnológica, Franca: Uni-FACEF, v. 15, n. 1, p. 180-209, 2025.

FERKO, Enxhi; BUCAIONI, Alessio; BEHNAM, Moris. Architecting Digital Twins. IEEE Access, Västerås: Mälardalen University, 2021/2022.

RICHARDS, Mark; FORD, Neal. Fundamentals of Software Architecture: An Engineering Approach. 1. ed. Boston/Sebastopol: O'Reilly Media, 2020.

SIKORA, Rostyslav; POSTOLIUK, Andrii. Comparative Analysis of the Effectiveness of Architectural Styles REST, GraphQL, and gRPC for Scalable Microservice Systems. Herald of Khmelnytskyi National University, Khmelnytskyi: State University of Trade and Economics / Ternopil Ivan Pulyk National Technical University, n. 4 (355), p. 560-568, 2025. DOI: <https://doi.org/10.31891/2307-5732-2025-355-79>.

VALE, Guilherme et al. Designing Microservice Systems Using Patterns: An Empirical Study on Quality Trade-Offs. Porto/São Paulo/Stuttgart/Bolzano: Faculdade de Engenharia da Universidade do Porto (FEUP), Instituto de Matemática e Estatística da USP, University of Stuttgart, Free University of Bozen-Bolzano, 2021.

VELEPUCHA, Victor; FLORES, Pamela. A survey on microservices architecture: Principles, patterns and migration challenges. IEEE Access, Quito: Escuela Politécnica Nacional, 2017.